Informatik 1: Grundlagen der Programmentwicklung

EVA-Prinzip, Daten und Information, Programmierprozess

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. EVA Prinzip
2. Daten-Informationen-Wissen
3. Programmierprozess – von der Aufgabe zum Code

TL;DL – Kurzfassung

- **Daten-Informationen-Wissen:** Daten sind Werte, Informationen sind Daten mit Kontext, Wissen ist Informationen mit Interpretation
- **EVA-Prinzip:** Eingabe, Verarbeitung, Ausgabe – das Grundmuster jedes Informationssystems
- **Programmierprozess:** Erst verstehen, planen und von Hand lösen – Code kommt zuletzt

1. EVA-Prinzip: Computersystem

Eingabe → **V**erarbeitung → **A**usgabe

Komponente	Beispiele
Eingabe	Tastatur, Maus, Touchpad, Scanner
Verarbeitung	CPU, Chipsatz, Controller
Ausgabe	Monitor, Lautsprecher, Drucker
Speicher	RAM, Festplatte, SSD, USB-Stick

1. EVA-Prinzip: Programm/Prozess

Input ⟶ Blackbox (Verarbeitung) ⟶ Output

Beispiel: OEE-Berechnung

Schritt	Inhalt
Eingabe	Rohdaten aus Produktionssimulation (Logdateien)
Verarbeitung	Berechnung in Excel/Python
Ausgabe	Verfügbarkeit 0,646 / Leistung 0,849 / Qualität 0,959 / OEE 52,6%

2. Daten, Information, Wissen

Drei Ebenen

Ebene	Beschreibung	Beispiel
Daten (Syntaktik)	Zeichen + Syntax	181
Information (Semantik)	Daten + Bedeutung	181 cm ist die Größe von Herrn Mustermann
Wissen (Pragmatik)	Information + Kontext	Durchschnittsgröße zur Parkbank-Konstruktion

Daten sind Rohwerte. Information entsteht durch Interpretation. Wissen entsteht durch Anwendung.

2. Daten vs. Information: ISO-Norm

ISO IEC 2382-1 (1993)

Daten: Wieder interpretierbare Darstellung von Informationen in formalistischer Art, geeignet zur Kommunikation, Interpretation und Verarbeitung. Inhalt in Zeichen kodiert (Syntax).

Information: Aus Daten gewonnen durch Interpretation in einem Bedeutungszusammenhang (Semantik).

Signal: Physikalische Darstellung einer Information – ein oder mehrere Parameter tragen Information über variable Größen.

2. Daten-Information-Wissen: Stufenmodell

Aufsteigende Veredelung

- **1.20** $\longrightarrow$ Daten (Rohwert)
- **Devisenkurs: 1 Euro = 1.20 Dollar** $\longrightarrow$ Information (Kontext)
- **Auswirkungen auf Zinsen und Aktienkurse** $\longrightarrow$ Wissen (Interpretation)

Information = Daten + Zweckbezug
Wissen = Information + Interpretation/Klassifikation

2. Wissenstreppe nach North

Von Zeichen zur Wettbewerbsfähigkeit

Zeichen $\rightarrow$ (+Syntax) **Daten** $\rightarrow$ (+Bedeutung) **Informationen** $\rightarrow$ (+Vernetzung) **Wissen** $\rightarrow$ (+Anwendung) **Handeln** $\rightarrow$ (+richtig handeln) **Kompetenz** $\rightarrow$ (+Einzigartigkeit) **Wettbewerbsfähigkeit**

Bereich	Umfang
Informationsmanagement	Zeichen bis Informationen
Wissensmanagement	Wissen bis Wettbewerbsfähigkeit
Explizites Wissen	Kodifizierbar, teilbar
Implizites Wissen	Erfahrung, schwer übertragbar

3. Programmierprozess: Warum?

Häufige Anfängerfehler

- Sofort Code schreiben ohne nachzudenken
- Kein Plan, was das Programm eigentlich tun soll
- Keine Testfälle – “läuft irgendwie” reicht nicht
- Bei Fehlern alles löschen und von vorne anfangen

Programmieren ist Problemlösen. Und Problemlösen folgt einem Ablauf: erst verstehen, dann planen, dann umsetzen, dann prüfen.

3. Programmierprozess: Überblick

Von der Aufgabe zum Code in 6 Schritten

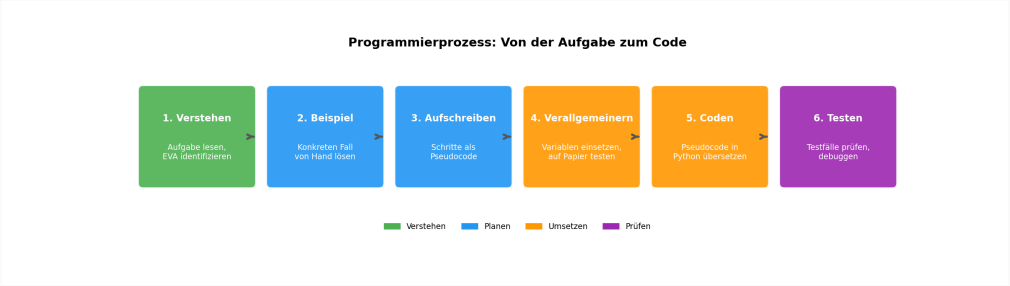


Abbildung 1: Programmierprozess

3. Schritt 1: Aufgabe verstehen

Was ist überhaupt gefragt?

- Aufgabenstellung **vollständig lesen** (wirklich!)
- In **eigenen Worten** wiedergeben: Was soll das Programm tun?
- **EVA-Prinzip** anwenden:
  - Was ist die **Eingabe**? (Welche Daten bekommt das Programm?)
  - Was ist die **Ausgabe**? (Was soll am Ende rauskommen?)
  - Was passiert bei der **Verarbeitung**? (Welche Regeln/Berechnungen?)

Wer die Aufgabe nicht erklären kann, kann sie auch nicht programmieren.

3. Schritt 2: Beispiel von Hand lösen

Was soll am Ende herauskommen?

- Ein **konkretes Beispiel** nehmen und **manuell** durchrechnen
- Mit echten Zahlen/Werten arbeiten, nicht abstrakt denken
- Das Ergebnis aufschreiben – das ist euer Testfall!

Beispiel: “Schreibe eine Funktion, die prüft ob eine Zahl gerade ist”

- Eingabe: 4 $\rightarrow$ Ergebnis: ja (4 / 2 = 2, kein Rest)
- Eingabe: 7 $\rightarrow$ Ergebnis: nein (7 / 2 = 3, Rest 1)
- Eingabe: 0 $\rightarrow$ Ergebnis: ja (0 / 2 = 0, kein Rest)

3. Schritt 3: Schritte aufschreiben

Den Lösungsweg in Worten festhalten

Frage: **Was habe ich gerade getan?** Jeden Schritt einzeln notieren.

- Noch kein Python – nur menschliche Sprache!
- Alternativ: Ablaufdiagramm zeichnen (UML)

3. Schritt 3: Pseudocode-Beispiel

```
Nimm eine Zahl entgegen
Teile die Zahl durch 2
Wenn der Rest gleich 0 ist:
    Gib "gerade" zurück
Sonst:
    Gib "ungerade" zurück
```

Pseudocode beschreibt die Logik – die Programmiersprache kommt später.

3. Schritt 4: Verallgemeinern

Vom Einzelfall zur allgemeinen Lösung

- Konkrete Zahlen durch **Variablen/Parameter** ersetzen
- Prüfen: Funktioniert der Pseudocode auch mit **anderen Eingaben**?
- Algorithmus **auf dem Papier testen** (Tracing):

Tracing mit verschiedenen Eingaben

zahl	zahl % 2	Rest == 0?	Ergebnis
4	0	ja	“gerade”
7	1	nein	“ungerade”
0	0	ja	“gerade”
-3	1	nein	“ungerade”

3. Schritt 5: In Code übersetzen

Vom Pseudocode zur Programmiersprache

Jetzt erst wird programmiert! Pseudocode Zeile für Zeile übersetzen:

```
def ist_gerade(zahl):
    if zahl % 2 == 0:
        return "gerade"
    else:
        return "ungerade"
```

3. Schritt 5: Tipps zur Übersetzung

Eine Zeile Pseudocode = eine Zeile Code (ungefähr). Wenn eine Zeile nicht übersetzbar ist $\longrightarrow$ Pseudocode verfeinern.

Nicht alles auf einmal – Funktion für Funktion entwickeln und testen.

Erst Pseudocode schreiben, dann übersetzen. Nie direkt drauflos programmieren.

3. Schritt 6: Testen

Tut das Programm, was es soll?

Testfälle aus Schritt 2 verwenden und Ergebnisse mit Erwartungen abgleichen:

```
print(ist_gerade(4))   # erwartet: "gerade"
print(ist_gerade(7))   # erwartet: "ungerade"
print(ist_gerade(0))   # erwartet: "gerade"
```

Stimmt das Ergebnis nicht? → Systematisch debuggen (siehe nächste Folie)

3. Schritt 6: Debuggen

Bei Fehlern: Systematisch vorgehen

1. **Fehlermeldung lesen** – Was sagt Python?
2. **Eingabe prüfen** – Stimmen die Testdaten?
3. **Tracing wiederholen** – Schritt für Schritt durchgehen
4. **Nicht alles löschen!** – Fehler isolieren und beheben

3. Der Prozess im Überblick

Erst denken, dann tippen. Die meiste Arbeit passiert *vor* dem Code.

1. **Verstehen** – Aufgabe lesen, EVA identifizieren
2. **Beispiel** – Konkreten Fall von Hand lösen
3. **Aufschreiben** – Schritte als Pseudocode notieren
4. **Verallgemeinern** – Variablen einsetzen, auf Papier testen
5. **Coden** – Pseudocode in Python übersetzen
6. **Testen** – Testfälle durchlaufen, debuggen

Inspiriert von Polya (1945) und dem “Seven Steps”-Ansatz (Duke University, Hilton et al.)

3. Häufige Fehler und Tipps

Fehler	Besser
Sofort Code schreiben	Erst Pseudocode, dann Code
“Läuft irgendwie”	Mit Testfällen aus Schritt 2 prüfen
Alles in eine Funktion	Teilprobleme einzeln lösen
Bei Fehler alles löschen	Fehlermeldung lesen, Fehler isolieren
Nur einen Fall testen	Normalfall, Grenzfall, Fehlerfall testen
Code kopieren ohne Verständnis	Erst von Hand lösen, dann programmieren

Zusammenfassung

- **EVA:** Eingabe $\rightarrow$ Verarbeitung $\rightarrow$ Ausgabe – Grundmuster aller Informationssysteme
- **Daten** $\rightarrow$ **Information** $\rightarrow$ **Wissen:** Veredelung durch Kontext und Interpretation
- **Programmierprozess:** Verstehen $\rightarrow$ Beispiel $\rightarrow$ Pseudocode $\rightarrow$ Verallgemeinern $\rightarrow$ Code $\rightarrow$ Testen